Veer D2 Detailed Design

Course: Computer Science Senior Design I

Advisor: Dr. Jillian Aurisano

Team: Grady Algire, Brady Cooper, Briar Elliot, Dominic Rowland, Aiden Ward

Design version: 1.2 | October 13, 2026

Project goal: Convert driver speech into routing constraints for hands-free navigation

D2 purpose: D2 finishes every component that D1 left open: C1 Mobile Navigation, C4 Usage Monitor, and the glue components (authentication, notifications, configuration, logging). It also adds edge cases and error handling, a requirements traceability table, a design rationale, and our peer design review note. D2 closes the design phase.

Traceability basis: US-01 through US-05; AC-01.1 through AC-02.2; D0 decisions D01 through D10

1. Header, Conventions, and D2 Overview

Every D2 figure shows the project title and goal statement at the top and a legend at the bottom. D2 keeps all D0 and D1 conventions and adds the ones below for the new diagram types.

Carried over from D0/D1. C labels a component the team builds; E labels an external dependency. In block and data-flow diagrams, solid blue boxes are team components, and dashed gray boxes are external dependencies or actors. An arrow shows who starts the call; the reply comes back on the same interface. In data models, rectangles are entities or records, PK is a primary key, FK is a foreign key, and 1 / * show cardinality. Types follow D0: string = UTF-8, integer = whole number, number = decimal, Boolean = true/false, instant = ISO 8601 UTC, [] = array,? = optional. Durations are in seconds and distances are in meters.

New in D2: state machines. Rounded blue boxes are states. Red boxes are error states. A solid dark arrow is the happy path; a gray arrow is an alternate path (like a driver saying "no"), and a red dashed arrow is an error path. Arrow labels name the event that causes the change, plus any time limit. Figure 1 is our error-path diagram for C1.

New in D2: ID prefixes. Each edge case has an ID (EC-xx) that the traceability table and rationale point back to. Each rationale entry has an ID (R-xx). Peer review action items using PR-xx.

1.1 Two-minute overview

What Veer is. Veer is a phone navigation app that you control by talking. The driver says something like "I'm hungry for McDonald's" or "avoid the rain," and Veer turns that into a routing rule, plans a new route, reads back a short summary, and only switches routes after the driver says "okay." Raw audio is never saved, and passengers can mute the mic with one tap.

What D0 set up. D0 split Veer into four team components and four outside services. C1 Mobile Navigation is an Android app on the phone. C2 Speech Understanding turns speech into a typed constraint. C3 Route Planning turns constraints into a route proposal. C4 Usage Monitor counts AI, Maps, and weather calls per trip. The outside services are the phone itself (E1), the AI provider (E2), Google Maps (E3), and the weather provider (E4). D0 also defined interfaces I1 through I9.

What D1 covered. D1 detailed C2 and C3, since they are the core of the product. It added the data model for trips, constraints, and route proposals, the algorithm that checks AI output against an allow-list, the filter-then-rank route selection algorithm, the I2 and I4 API contracts, and the tech stack: Android/Kotlin, Node.js/TypeScript with Express, PostgreSQL, OpenAI, Google Maps, Open-Meteo, and Render.

What D2 adds. D2 finishes C1 (the trip state machine that owns the live route and the mic) and C4 (usage storage and the developer report). It also gives the same treatment to the glue pieces D0 only mentioned: authentication, notifications, configuration, and logging. D2 then adds one edge-case table across every component, a traceability table for every requirement, the design rationale, and the peer review notes.

1.2 Known gaps and open questions

These are the things we know are not settled yet. We list them here, so reviewers do not have to find them. Owners follow the D0 component owners.

ID	Gap or open question	Owner	Plan
G-01	How does the phone recognize "Veer, stop listening" without the cloud? D0 requires local mute, but we have not picked an on-device keyword detector.	Grady Algire	Test Android on-device speech recognition for the one phrase; fall back to tap-only mute if it cannot meet the 1 s limit.
G-02	Exact rain threshold, weather data freshness window, and max weather detour.	Dominic Rowland	Pick values from test drives; put them in configuration, not code.
G-03	OpenAI no-retention setting for audio is not verified yet. This blocks using E2 in a car demo.	Briar Elliot	Confirm the setting in writing before the first in-car test.
G-04	Google Maps terms for turn-by-turn display in our own app are not verified.	Brady Cooper	Review terms; if not allowed, hand off guidance to the Google Maps app.
G-05	How long C4 keeps usage counts.	Aiden Ward	Propose a retention period in the C4 section of D2.
G-06	Android may pause the mic or network when the app is in the background.	Grady Algire	Run C1 as a foreground service with a visible notification during a trip.
G-07	Actual monthly spending caps for AI, Maps, and weather.	Aiden Ward	Team agrees on caps; C4 enforces them through configuration.
G-08	Audio encoding and wire format for I2/I3.	Briar Elliot	Match whatever the chosen OpenAI streaming endpoint accepts.

2. Finished Components (the D2 diagrams)

D1 finished C2 and C3. This section finishes everything else: C1 Mobile Navigation, C4 Usage Monitor, and the glue components. Each part gives a data model, its behavior, and its interface.

2.1 C1 Mobile Navigation (owner: Grady Algire)

C1 is the Android app. It is the only component allowed to change the live route or control the mic. It talks to the driver through E1 (I1), sends speech to C2 (I2), and asks C3 for route proposals (I4).

Data model.

No persistent data. C1 keeps one TripSession in phone memory and clears it when the trip ends. Nothing is written to disk, including audio. The shared types (RoutePlan, RouteProposal, Constraint, LocationFix) are the same ones defined in D0 and used by C2 and C3.

Record	Field	Type	Meaning
TripSession	tripId	string	Set when the trip starts; sent on every I2/I4 call.
	tripRevision	integer	Goes up by 1 each time a route is activated. Replies with an older revision are ignored.
	state	enum	Idle, Listening, Interpreting, Planning, AwaitingApproval, Activating, Recovery, Muted, MuteError, Ended (Figure 1).
	destination	LatLng	Chosen before driving starts.
	activeRoute	RoutePlan	The route the driver is following now.
	activeConstraints	Constraint[]	Rules in effect for this trip. Cleared at trip end.
	pendingProposal	RouteProposal?	A proposal waiting for "okay" or "no."
	pendingRequestId	string?	The one I2 or I4 request C1 is waiting on. Only one at a time.
	micState	enum	listening, muted, or error. The on-screen indicator reads this field.
	lastLocation	LocationFix	Most recent GPS fix from E1.
AudioBuffer	frames	PCMFrame[]	In memory only. Holds the current utterance, capped at 15 s. Cleared on send, cancel, or mute.

C1's behavior is a state machine, shown in Figure 1. The figure includes the error paths. Every error path goes through Recovery, which speaks a short prompt and keeps the current route.

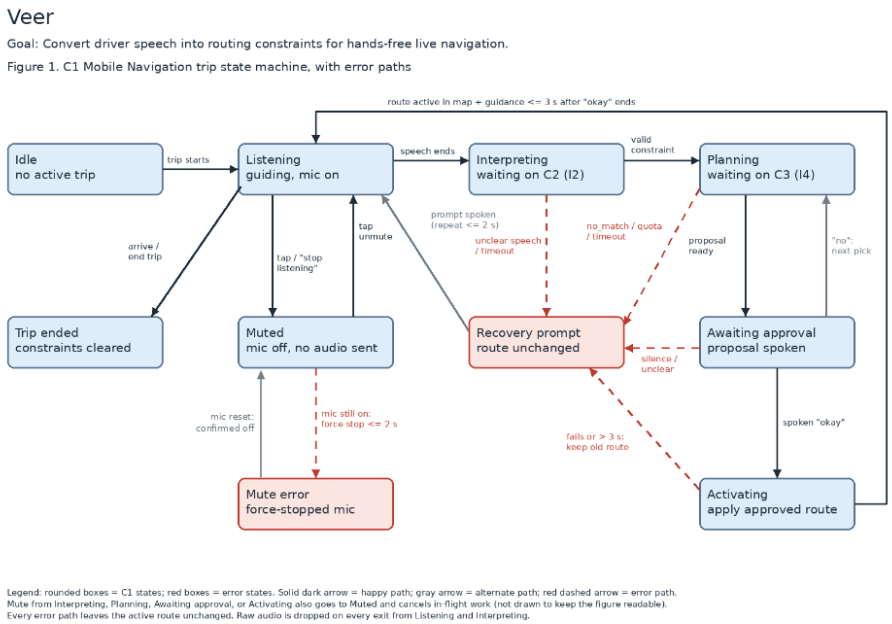


Figure 1. C1 trip state machine with error paths. Source: D2_C1_State_Machine.drawio.

The table below lists each transition so it can be coded and tested one row at a time.

From	Event	Action	To
Idle	Driver picks destination and starts trip	Create TripSession; tripRevision = 0; request first route (I4); start mic and location	Listening
Listening	End of speech detected	Send AudioBuffer on I2; clear buffer	Interpreting
Interpreting	C2 returns propose + valid constraint	Send constraints + trip context on I4	Planning
Interpreting	unclear_speech, invalid_audio, or timeout	Drop transcript; speak "Sorry, say that again?" within 2 s	Recovery
Planning	C3 returns RouteProposal (matching revision)	Store pendingProposal; speak confirmationText (15 words or fewer)	AwaitingApproval
Planning	no_match, quota, weather_unavailable, or timeout	Speak a short recovery prompt	Recovery
Planning / Interpreting	Reply with old tripRevision or wrong requestId	Ignore the reply	(no change)
AwaitingApproval	Driver says "okay"	Note approval end time; start 3 s timer	Activating
AwaitingApproval	Driver says "no"	Ask C3 for the next candidate (I4)	Planning
AwaitingApproval	Silence or unclear reply	Drop pendingProposal; keep route	Recovery
Activating	Route shown in map and guidance within 3 s	activeRoute = proposal route; tripRevision + 1	Listening
Activating	Apply fails or takes more than 3 s	Keep old route; say the change failed	Recovery
Recovery	Prompt finished	Clear pending request	Listening
Any listening-path state	Tap mute or "Veer, stop listening"	Close send gate; clear AudioBuffer; cancel in-flight I2/I4; stop mic	Muted
Muted	Mic still on after 1 s	Force-stop the audio process; show error	MuteError
MuteError	Mic reset and confirmed off	Clear error; show muted indicator	Muted
Muted	Tap unmute	Restart mic; show listening indicator	Listening
Listening	Arrive or end trip	Clear TripSession and AudioBuffer	Ended

Interface.

C1's outside interfaces are I1 (with E1), I2 (to C2), and I4 (to C3). I2 and I4 are already specified in D1 Section 5. Inside the app, the state machine is driven by the Kotlin functions below. Each returns a Result, so failures come back as typed errors instead of crashes.

Signature	Returns	Errors
startTrip(destination: LatLng)	TripSession	no_location (GPS not ready), mic_permission_denied, route_unavailable
onUtteranceEnded(frames: PCMFrame[])	Unit	busy (a request is already pending; utterance dropped), utterance_too_long (over 15 s)
onApprovalReply(intent: enum(accept, reject, unknown), endedAt: instant)	Unit	no_pending_proposal, activation_timeout
mute(source: enum(tap, localVoice))	MicState	mic_stop_failed (moves to MuteError)
unmute()	MicState	mic_permission_denied
endTrip()	Unit	none; always clears the session

D0 interface revisit (I1).

D0's I1 grew too large. It mixed mic control, location, speech output, mute events, destination entry, and approval replies in one interface with nine fields. D2 splits it into three smaller interfaces. The fields and behavior are the same; only the grouping changes. D0 is not edited; the change is recorded in the design rationale.

New ID	Covers	Fields
I1a Mic	Mic start/stop and audio frames	micControl: enum(start, stop, forceStop); audio: PCMFrame[]; micState: enum(listening, muted, error); mute: MuteEvent
I1b Location	GPS updates	locationControl: enum(start, stop); location: LocationFix
I1c Driver I/O	Spoken output, screen, and destination entry	spokenText: string; destination: LatLng?

Approval replies (accept or reject) were also listed on I1 in D0. They come back from C2 over I2, since C2 interprets the driver's "okay" or "no," so D2 removes them from I1.

2.2 C4 Usage Monitor

C4 counts AI, Maps, and weather calls for each trip. It checks monthly spending caps before a paid call and builds the developer report. It stores counts and costs only. It never stores audio, transcripts, or locations.

Data model.

C4 keeps three records in the PostgreSQL database.

Record	Field	Type	Meaning
UsageEvent	eventId (PK)	string	Unique per call.
	tripId (FK)	string	Trip that made the call.
	service	enum	ai, maps, or weather.
	operation	string	Call type, such as route_matrix.
	occurredAt	instant	When the call finished.
	units	integer	Billable units used.
	estCost	number	Estimated cost in USD.
	outcome	enum	ok, error, or throttled.
TripUsage	tripId (PK, FK)	string	One row per trip.
	aiCalls, mapsCalls, weatherCalls	integer	Running count per service.
	estCostTotal	number	Sum of estCost for the trip.
MonthlyBudget	service, month (PK)	enum, string	For example ai, 2026-10.
	used	number	Cost so far this month.
	state	enum	OK, Warning, or Exhausted.

One trip has one TripUsage row and many UsageEvent rows. MonthlyBudget is not tied to a trip.

Retention (closes G-05). Proposed: keep UsageEvent for 90 days, TripUsage for 12 months, and MonthlyBudget with no limit. MonthlyBudget holds no trip data. A nightly job deletes expired rows.

Spending caps (closes G-07). The cap for each service is a configuration value (Section 2.3.3). The team sets the amounts. C4 only enforces them.

Behavior.

C4 is a quota check before each paid call and a usage record after it. Budget states are OK, Warning at 80 percent, and Exhausted at 100 percent.

Step	Action
1	Before calling E2, E3, or E4, C2 or C3 calls checkQuota(service).
2	C4 compares MonthlyBudget.used to the cap from configuration.
3	Below 80 percent: allow. From 80 to 100 percent: allow and send a developer alert once. At 100 percent: deny.
4	On deny, the caller returns quota. C1 handles it in Recovery.
5	After the call, the caller sends recordUsage. C4 saves the event and updates both totals in one transaction.
6	The budget update is one atomic statement that refuses to pass the cap, so two calls at once cannot both slip through.
7	If the database is down, recordUsage queues events in memory (up to 500) and retries every 30 s. checkQuota allows the call, so navigation keeps working.
8	On the first of the month, a new MonthlyBudget row starts at OK.

Interface.

The developer report is for developers only and uses the admin key from Section 2.3.1.

Signature	Returns	Errors
checkQuota(service)	allow, allow_warn, or deny	unknown_service; usage_store_unavailable (treated as allow)
recordUsage(tripId, service, operation, units, outcome)	Unit	invalid_trip; usage_store_unavailable (queued)
GET /usage/report?from=&to=&groupBy=	Counts and cost per trip, day, or service	400 invalid_range; 401 unauthorized; 403 forbidden; 503 usage_store_unavailable

2.3 Glue components: authentication, notifications, configuration, logging

D0 mentioned these four pieces but gave them no detail. Each one is finished below with a data model, behavior, and interface.

2.3.1 Authentication

Veer has no driver accounts. Each app install is anonymous. Authentication only keeps strangers from spending our AI and Maps budget.

Data model.

Record	Field	Type	Meaning
Installation	installId (PK)	string	Random ID made at first launch.
	createdAt	instant	When the install registered.
	status	enum	active or revoked.
	refreshTokenHash	string	Hash only. The raw token is never stored.

Provider keys (OpenAI, Google Maps, weather) stay on the server. The app never holds them. The app keeps its refresh token in the Android Keystore. The developer report uses a separate admin key kept in server secrets.

Behavior.

Step	Action
1	First launch: the app calls POST /auth/register. The server returns a 1 hour access token and a refresh token.
2	Every call to C2, C3, and C4 sends the access token.
3	On token_expired, the app refreshes once and retries the call once.
4	If refresh fails, C1 goes to Recovery and says "Please reconnect." The current route stays.
5	A revoked install gets token_revoked on every call.
6	Registration is limited per IP address.

Interface.

Endpoint	Returns	Errors
POST /auth/register	accessToken, refreshToken, expiresIn	429 rate_limited; 503 auth_unavailable
POST /auth/refresh	accessToken, expiresIn	401 invalid_refresh; 403 token_revoked

2.3.2 Notifications

There are two kinds: notices for the driver and alerts for the team.

Data model.

Phone: no persistent data. Server: one record, AlertState, keyed by service, month, and level, with a lastSentAt time. It stops the same alert from going out twice.

Behavior.

Kind	When	What happens
Foreground notification	Whole trip (G-06)	Shows "Veer is navigating," mic state, and a Mute button. Keeps Android from pausing the mic.
Driver notice	Mute error, failed route change, offline	Spoken and shown through I1c.
Developer alert	Budget reaches 80 and 100 percent	Sent by email or Slack, once per service, month, and level.

Interface.

Signature	Returns	Errors
notifyDriver(kind, text)	Unit	notification_permission_denied (falls back to spoken only)
sendDeveloperAlert(service, level, percentUsed)	Unit	alert_channel_unavailable (logged, retried on the next event)

2.3.3 Configuration

Values we expect to change after test drives live in configuration, not in code.

Data model.

Key and value settings. Secrets come from Render environment variables. Everything else comes from one versioned config file. The phone caches the client subset.

Key	Used by	Value
mute.forceStopSec	C1	1
recovery.promptSec	C1	2
activation.maxSec	C1	3
utterance.maxSec	C1	15
confirmation.maxWords	C2	15
timeout.interpretSec, planSec, approvalSilenceSec	C1	Set in testing
weather.rainThreshold, maxAgeSec, maxDetourSec	C3	Set from test drives (G-02)
budget.cap.ai, maps, weather	C4	Team sets (G-07)
retention.eventDays, tripMonths	C4	90, 12

Behavior.

Step	Action
1	At startup, the backend loads every key and checks it against a schema (type and range).
2	If a key is missing or invalid, the backend refuses to start and logs config_invalid.
3	Changes need a redeploy on Render.
4	At trip start, the app calls GET /config/client and caches the result.
5	If that call fails, the app uses its cached copy, then the defaults built into the app.

Interface.

Endpoint	Returns	Errors
GET /config/client	version, client values (never secrets)	401 unauthorized; 503 config_unavailable (app uses its cache)

2.3.4 Logging

Server logs are JSON lines on Render, kept for 30 days. The phone writes logs to Logcat in debug builds only. Release builds write nothing to disk.

Data model.

Field	Type	Meaning
ts	instant	When the line was written.
level	enum	debug, info, warn, or error.
component	enum	C1 to C4, auth, or config.
tripId, requestId	string?	Lets us trace one request across components.
event	string	Short code, such as route_activated.
errorCode	string?	Same names used in this document.
durationMs	integer?	Time taken.

Never logged: audio, transcripts, raw GPS coordinates, tokens, and API keys.

Behavior.

Step	Action
1	Each request gets a requestId at the first server call. C2, C3, and C4 pass it along.
2	Errors log at error level with their errorCode. Expected events, such as the driver saying "no," log at info.
3	A redaction filter removes the banned fields before any line is written.
4	If the log destination is down, the app keeps working and drops the lines.

Interface.

Signature	Returns	Errors
log(level, component, event, fields)	Unit	None. It never throws.

3. Edge Cases and Error Handling

Veer must continue providing navigation when individual components fail, prevent unintended route changes, and protect passenger privacy. Every component has defined handling for boundary values, timing and concurrency, bad or missing input, and external dependencies.

The table below identifies each edge case, the expected system behavior, the responsible handler, and whether the error is recoverable.

3.1 Edge Case Table

ID	Component	Category	Edge case	Design decision / handler / recoverable
EC-01	C1 Mobile Navigation	Boundary values	Audio exceeds 15 seconds.	Drop utterance and clear buffer. Handler: onUtteranceEnded. Recoverable: Yes.
EC-02	C1 Mobile Navigation	Timing/concurrency	Outdated proposal arrives after a newer request.	Ignore old tripRevision or incorrect requestId. Handler: C1 state machine. Recoverable: Yes.
EC-03	C1 Mobile Navigation	Bad/missing input	Trip starts without valid destination or GPS.	Return no_location; do not begin until valid location is available. Handler: startTrip. Recoverable: Yes.
EC-04	C1 Mobile Navigation	External dependencies	Android microphone fails to stop when muted.	Block outgoing audio immediately; clear buffer; force-stop microphone and enter MuteError. Handler: mute. Recoverable: Yes, after mic reset. Verify 2-second deadline.
EC-05	C2 Speech Understanding	Boundary values	Confirmation exceeds 15 words.	Do not cut the sentence off. Replace it with a short summary built from the proposal (place and added minutes) that fits in 15 words. The route proposal is unchanged and still needs "okay" before it activates. Handler: C2 output validator. Recoverable: Yes.
EC-06	C2 Speech Understanding	Timing/concurrency	AI interpretation exceeds timeout.	Cancel request; return timeout; keep route. Handler: C2 timeout handler. Recoverable: Yes.
EC-07	C2 Speech Understanding	Bad/missing input	Audio empty, corrupted, or unparseable.	Return invalid_audio or unclear_speech; C1 requests repetition within 2 seconds. Handler: C2 validator/C1 Recovery. Recoverable: Yes.

EC-08	C2 Speech Understanding	External dependencies	OpenAI unavailable or quota exhausted.	Return provider/quota error; keep route. Handler: C2 provider adapter. Recoverable: Yes, once restored.
EC-09	C3 Route Planning	Boundary values	No matching food stop within 10-minute detour.	Return no_match; preserve route; prompt for another place within 3 seconds. Handler: C3 filter-then-rank/C1 Recovery. Recoverable: Yes.
EC-10	C3 Route Planning	Timing/concurrency	Planning times out or stale proposal arrives.	Return timeout or reject stale response. Handler: C3 planner/C1 revision validation. Recoverable: Yes.
EC-11	C3 Route Planning	Bad/missing input	Invalid destination or unsupported constraint.	Reject with validation error; do not propose route. Handler: C3 input validator. Recoverable: Yes.
EC-12	C3 Route Planning	External dependencies	Google Maps or weather service unavailable.	Keep active route; surface failure of requested change. Handler: C3 provider adapter. Recoverable: Yes.
EC-13	C4 Usage Monitor	Boundary values	Monthly cap reached.	Deny paid calls and alert developer. Handler: checkQuota. Recoverable: Yes, after cap reset/adjustment.
EC-14	C4 Usage Monitor	Timing/concurrency	Concurrent requests use remaining budget.	Atomic quota reservation/update to avoid overrun. Handler: C4 budget transaction. Recoverable: Yes; implementation needs verification.
EC-15	C4 Usage Monitor	Bad/missing input	Unknown service or invalid trip ID.	Reject unknown_service or invalid_trip. Handler: C4 validator. Recoverable: Yes.
EC-16	C4 Usage Monitor	External dependencies	PostgreSQL unavailable.	Queue up to 500 usage events; retry every 30 seconds; continue existing

				navigation. Handler: C4 storage fallback. Recoverable: Partially; full queue and cap enforcement unresolved.
EC-17	Authentication	Boundary values	Registration rate limit exceeded.	Return 429 rate_limited. Handler: auth rate limiter. Recoverable: Yes.
EC-18	Authentication	Timing/concurrency	Token expires during request.	Refresh token once and retry once. Handler: auth middleware/C1. Recoverable: Yes if refresh succeeds.
EC-19	Authentication	Bad/missing input	Refresh token invalid or install revoked.	Return 401/403; request reconnection; keep route. Handler: /auth/refresh. Recoverable: Yes after registration, unless revoked.
EC-20	Authentication	External dependencies	Auth backend unavailable.	Return 503; continue existing route without new requests. Handler: auth middleware. Recoverable: Yes.
EC-21	Notifications	Boundary values	Duplicate budget alert generated.	Suppress duplicate using AlertState. Handler: sendDeveloperAlert. Recoverable: Yes.
EC-22	Notifications	Timing/concurrency	Developer alert cannot be delivered.	Log and retry on next qualifying event. Handler: notification dispatcher. Recoverable: Yes.
EC-23	Notifications	Bad/missing input	Notification text/type missing.	Reject invalid notification and log. Handler: notification validator. Recoverable: Yes.
EC-24	Notifications	External dependencies	Android notification permission denied.	Fall back to spoken notice where possible. Handler: notifyDriver. Recoverable: Partially; foreground-service implications need testing.

EC-25	Configuration	Boundary values	Configuration numeric value out of range.	Reject configuration at startup. Handler: schema validator. Recoverable: Yes after correction.
EC-26	Configuration	Timing/concurrency	Configuration changes during active trip.	Keep trip snapshot; apply new values at next trip. Handler: C1 config cache. Recoverable: Yes.
EC-27	Configuration	Bad/missing input	Required config missing or wrong type.	Log config_invalid and refuse backend startup. Handler: config loader. Recoverable: Yes after correction.
EC-28	Configuration	External dependencies	/config/client unavailable.	Use cached values then defaults. Handler: C1 config loader. Recoverable: Yes.
EC-29	Logging	Boundary values	Prohibited sensitive fields in log.	Redact audio, transcripts, GPS, tokens, keys. Handler: redaction filter. Recoverable: Yes.
EC-30	Logging	Timing/concurrency	Multiple requests log simultaneously.	Write separate JSON records with request IDs. Handler: server logger. Recoverable: Yes.
EC-31	Logging	Bad/missing input	Optional log fields invalid/missing.	Omit optional fields, never throw. Handler: log. Recoverable: Yes.
EC-32	Logging	External dependencies	Render log destination unavailable.	Drop log lines; continue navigation. Handler: log. Recoverable: Partially; lost lines cannot be restored.
EC-33	C1 Mobile Navigation	External dependencies	Network lost during an active trip.	Keep guiding on the stored activeRoute. Voice requests time out and enter Recovery; say "No connection" once. Resume requests when the network returns. Handler: C1 Recovery. Recoverable: Yes, when the network returns.

EC-34	C1 Mobile Navigation	External dependencies	GPS signal lost during an active trip.	Keep guiding on activeRoute using lastLocation; say "GPS signal lost" once. Do not send new route requests (I4) until a fix returns. Handler: C1 location handling (I1b). Recoverable: Yes, when a fix returns.

3.2 Error Handling Flow

Figure 1, the C1 Trip State Machine from Section 2.1, shows how successful requests and failed requests move between states.

The general error-handling flow is:

1. C1 receives a voice command and sends the audio to C2 for interpretation.
2. C2 validates the input and converts it into a routing constraint.
3. C3 attempts to generate a valid route proposal.
4. If interpretation or planning fails, the component returns a typed error.
5. C1 enters Recovery, gives a short explanation, and preserves the active route.
6. If a valid proposal is returned, C1 enters AwaitingApproval.
7. The driver must explicitly say "okay" before Veer activates the proposed route.
8. If the driver rejects the proposal, Veer requests the next candidate. If the driver remains silent or gives an unclear response, Veer discards the proposal and retains the current route.
9. If route activation fails or takes longer than 3 seconds after approval, C1 keeps the previous route and enters Recovery.
10. The application returns to Listening after recovery, unless the microphone is muted or the trip has ended.

All component errors use consistent error codes and are handled without terminating an active navigation session whenever possible.

3.3 Error Handling Policies

Veer follows four main policies:

Navigation continuity: Failed requests must not replace the current route. Existing navigation remains active unless the trip is explicitly ended.

Privacy protection: Muting immediately blocks outgoing audio and clears the audio buffer. Microphone shutdown failures trigger a force-stop and visible error.

Controlled recovery: Authentication refresh is attempted once. Usage storage retries every 30 seconds. Other components use their documented timeouts and error responses.

Error visibility: Driver-facing errors use brief spoken or displayed messages. Developer-facing errors are written to structured logs without sensitive information.

3.4 Remaining Error-Handling Gaps

Some implementation details still require validation.

- G-01: Local voice recognition must be tested to verify that microphone muting meets the acceptance criteria.
- G-02: Weather thresholds, freshness limits, and acceptable detours must be finalized.
- G-03: OpenAI audio-retention settings must be verified before in-car testing.
- G-04: Google Maps usage terms must be reviewed.
- G-06: Android background microphone and network behavior must be tested.
- G-07: Monthly spending caps must be selected and tested.
- G-08: Audio encoding and message formats must be finalized.

The team must also finalize behavior when C4's in-memory usage queue reaches capacity. Because allowing paid calls while usage storage is unavailable can conflict with strict budget limits, quota enforcement during database outages requires additional verification.

4. Requirements Traceability Table

This table connects Veer's functional requirements, acceptance criteria, and non-functional requirements to their corresponding design elements in D0, D1, and D2.

Each row identifies the requirement, its priority, the specific interface, algorithm, entity, or figure supporting it, and how the design satisfies the requirement.

4.1 Functional Requirements

Requirement	Priority	Design elements	How satisfied

US-01 — Add food stop by voice	Must	C2 add_waypoint; C3 filter-then-rank; I2/I4; RouteProposal	C2 parses requested food stop. C3 uses Google Maps to find candidates within acceptable detour; C1 requests explicit approval.
US-02 — Avoid rain	Should	C3 route ranking; E4 Open-Meteo; weather.rainThreshold/maxDetourSec	C3 evaluates route weather and proposes lower-rain alternatives. Thresholds remain open (G-02).
US-03 — View AI and Maps calls per trip	Must	UsageEvent; TripUsage; recordUsage; GET /usage/report	C4 associates calls with trip IDs and reports counts/costs.
US-04 — Show listening status and let passengers mute	Must	I1a; micState; mute(); AudioBuffer; Figure 1	Mic indicator is visible; tap/local voice mute shuts audio gate and clears buffer.
US-05 — Confirm interpretation before changing route	Must	RouteProposal.confirmationText; AwaitingApproval; onApprovalReply; Figure 1	C1 reads back proposal and applies it only after explicit “okay”; silence leaves route unchanged.

4.2 Acceptance Criteria

Requirement	Priority	Design elements	How satisfied / gap
AC-01.1 — McDonald’s stop ≤10-min detour; ≤15-word confirmation within 3 seconds	Must	C2 add_waypoint; C3 filter-then-rank; confirmation.maxWords; I4	Route candidate is filtered by detour, then confirmed. Verify 10-min threshold and 3-second response; D2 requires separate explicit approval before activation.
AC-01.2 — Zero raw audio files on phone/server	Must	AudioBuffer; C1 in-memory storage; logging redaction	Clear raw audio and do not write audio files; verify external AI provider retention (G-03).

AC-01.3 — No match ≤10 min: keep route, prompt within 3 seconds	Must	C3 no_match; C1 Recovery; recovery.promptSec; EC-09	Return no_match, retain route, prompt for a different place; verify response time.
AC-01.4 — Unclear speech: keep route, prompt within 2 seconds	Must	C2 validation; unclear_speech; C1 Recovery; EC-07	Reject unclear constraint; keep route; speak retry prompt within 2 seconds.
AC-02.1 — Voice mute within 1 second; show muted	Must	I1a; mute(); micState; mute.forceStopSec	Close send gate, stop mic, update indicator; verify local phrase recognition (G-01).
AC-02.2 — Force-stop by 2 seconds; no post-mute audio sent/stored	Must	mute(); MuteError; AudioBuffer; EC-04; Figure 1	Immediately block sending and clear buffers; force-stop process if required; test timing.

4.3 Non-Functional Requirements

Requirement	Priority	Design elements	How satisfied / gap
NFR-01 — Protect passenger/driver privacy	Must	AudioBuffer; logging redaction; mute()	Raw audio not intentionally persisted by Veer; logs exclude private values; provider policy remains G-03.
NFR-02 — Preserve navigation on failures	Must	C1 Recovery; activeRoute; Figure 1; EC-01–EC-32	Failed changes retain active route.
NFR-03 — Secure third-party credentials	Must	/auth/register; /auth/refresh; Android Keystore	Backend holds external API keys; client stores refresh token securely.
NFR-04 — Control API spending	Must	MonthlyBudget; checkQuota; recordUsage	Track calls, enforce caps, alert at 80%/100%; verify outage/concurrency behavior.
NFR-05 — Debug without private data	Should	JSON logger; requestId; errorCode; redaction filter	Correlate failures without audio/transcripts/GPS/tokens.

NFR-06 — Configurable system behavior	Should	Versioned config; /config/client; schema	Configure timeouts, weather criteria, retention, and budget caps.
NFR-07 — Minimize driver distraction	Must	I1c; confirmation.maxWords; notifyDriver	Short spoken confirmations and voice approval reduce phone interaction.
NFR-08 — Limit usage retention	Should	UsageEvent; TripUsage; nightly cleanup	Proposed 90-day event, 12-month trip usage retention; needs approval.
NFR-09 — Support background navigation	Must	Android foreground service; notification; I1a	Foreground service with visible indication; test G-06.

4.4 Traceability Gaps

Gap	Requirements	Action
G-01	US-04; AC-02.1; AC-02.2	Test local “Veer, stop listening” detection and 1-second/2-second mute deadlines.
G-02	US-02; NFR-06	Set weather thresholds, freshness, and maximum detour.
G-03	AC-01.2; NFR-01	Verify AI provider audio retention settings.
G-04	US-01; US-02; NFR-02	Confirm Google Maps display/navigation usage terms.
G-06	NFR-09	Test Android background audio/network behavior.
G-07	US-03; NFR-04	Finalize service spending caps and enforcement.
G-08	US-01; US-05	Finalize I2/I3 audio format.
D2 approval alignment	US-01; US-05; AC-01.1	Update UC-01 so silence never approves; only “okay” activates a proposal.
D2 timing alignment	AC-01.1; AC-02.1; AC-02.2	Clarify 3-second initial confirmation versus 3-second post-approval activation; verify mute deadlines.

4.5 Reverse Traceability Check

Component	Requirements	Status
C1 Mobile Navigation	US-01, US-04, US-05, AC-02.1, AC-02.2	Covered
C2 Speech Understanding	US-01, US-05, AC-01.4	Covered
C3 Route Planning	US-01, US-02, AC-01.1, AC-01.3	Covered
C4 Usage Monitor	US-03, NFR-04, NFR-08	Covered
Authentication	NFR-03	Covered
Notifications	US-04, NFR-07, NFR-09	Covered
Configuration	US-02, NFR-06	Covered
Logging	NFR-01, NFR-05	Covered

Every major component has at least one supporting requirement. No component is currently identified as unnecessary scope.

The review also confirms that requirements are connected to specific data models, endpoints, algorithms, or diagrams rather than only component names.

The remaining gaps will become implementation and testing tasks after the design phase. These include weather thresholds, microphone shutdown validation, audio retention verification, and API spending enforcement.

5. Design Rationale

Each entry below is a significant D1 or D2 decision, written as "We chose X over Y because ___. We accepted the trade-off of ___ because ___." Decisions already made in D0 (D01 to D10) are not repeated; entries point to the D0 decision log where they build on it. The last column marks whether the peer design review changed the decision.

ID	Decision and trade-off	Traces to	Changed by review?
R-01	We chose filter-then-rank route selection over one weighted score for all routes because hard limits, like the 10-minute food-stop detour, must never be broken by a high score somewhere else. We accepted the trade-off of sometimes skipping a route that is just over a limit but better overall	D1 §3.2; AC-01.1; EC-09; D0 D10	No

	because a broken promise to the driver is worse than a slightly worse route, and pass/fail filters are easier to test.		
R-02	We chose to check every AI reply against an allow-list of constraint types over trusting the AI's output or letting it build routes because the AI can return bad or made-up values and must never control routing directly. We accepted the trade-off of needing a code change for every new kind of request because a short list we can test is safer while driving.	D1 §3.1; EC-05, EC-07; D0 D04	No
R-03	We chose a C1 state machine with one pending request and a tripRevision number over allowing several requests at once because overlapping replies could swap the route to an old proposal. We accepted the trade-off of dropping a second command spoken while one is still being handled (busy) because the driver can repeat it, while a wrong route switch is hard to undo.	Figure 1; TripSession; EC-02, EC-10	No
R-04	We chose to split D0's I1 into I1a Mic, I1b Location, and I1c Driver I/O, and move approval replies to I2, over keeping one nine-field I1 because the old interface mixed unrelated jobs and was hard to test. We accepted the trade-off of more interfaces to document because each one is now small and the mic path can be tested by itself for the 1-second mute rule. D0 was not edited; this entry records the change.	§2.1 I1 revisit; US-04; AC-02.1	No
R-05	We chose anonymous install tokens with keys kept on the server over driver accounts or putting provider keys in the app because Veer has no need for accounts and keys in an app can be pulled out and used to spend our budget. We accepted the trade-off of no history across devices and the chance that one person registers many installs because per-IP rate limits and C4 caps limit the damage.	§2.3.1; NFR-03; EC-17 to EC-20	No
R-06	We chose to let checkQuota allow calls when the usage database is down over blocking all paid calls because navigation must keep working when a side service fails. We accepted the trade-off of possibly going over a cap during an outage because outages should be short, events are queued and retried, and this gap is listed as open.	§2.2 step 7; EC-16; §3.4; D0 D08	No
R-07	We chose one atomic database update that refuses to pass the cap over a simple in-memory counter because two calls at the same time could both spend the last of the budget. We accepted the trade-off of an extra database write on every paid call because the write is small and PostgreSQL is already in the stack.	MonthlyBudget; checkQuota; EC-14; NFR-04	No

R-08	We chose to keep C4 usage events for 90 days and trip totals for 12 months over keeping everything forever because cost reports only need recent detail and less stored data means less privacy risk. We accepted the trade-off of losing per-call detail for older trips because monthly totals are kept with no limit and are enough for cost tracking. This closes G-05.	§2.2 retention; NFR-08; D0 D06	No
R-09	We chose a versioned config file plus Render environment variables, checked at startup, over a live settings service because our team is small and the values change only after test drives. We accepted the trade-off of needing a redeploy for every change, and of the backend refusing to start on a bad value, because failing fast is safer than running with a wrong timeout or cap.	§2.3.3; NFR-06; EC-25 to EC-28	No
R-10	We chose JSON server logs with a redaction filter and no logs saved on the phone in release builds over full request logging or a third-party crash tool because logs must never hold audio, transcripts, GPS, or tokens. We accepted the trade-off of harder debugging of phone-only bugs because the requestId still lets us trace a failure across C2 to C4, and debug builds still log to Logcat.	§2.3.4; NFR-01, NFR-05; EC-29; D0 D03	No

6. Peer Design Review Notes

Completeness (FIXED)

- Section 2.1 Figure 1 - No transition out of MuteError
- Section 3.1 - Could describe edge cases for network or GPS loss during active trip

Feasibility (FIXED)

- No notes

Clarity (FIXED)

- 3.1 EC-05 - "Shorten" could be made more descriptive, does it just get truncated? How would this affect route change

Risk

- No notes

Thank you,

Collin Barrett (Peer Reviwing Group Lead)